

Ros Day01 hw1 보고서

작성자: Doyeon Park

작성일: 2026.09.09

목차

1. 과제 목적
2. ROS Domain 설정
3. Turtlesim 실행
4. 원 그리기
5. 사각형 그리기
6. 삼각형 그리기
7. 문제점 및 해결

1. 과제 목적

ROS2 의 Topic 통신을 이용하여 turtlesim 의 거북이에게 속도 명령을 직접 전달하고, 이를 통해 원, 사각형, 삼각형을 그려보았다.

이번 과제에서는 키보드 조작이 아니라 터미널의 ros2 topic pub 명령어를 사용하여 .turtle1/cmd_vel Topic 에 속도 메시지를 직접 발행하였다.

2. ROS Domain 설정

다른 사람의 ROS2 노드와 통신이 겹치는 것을 방지하기 위하여 ROS Domain ID 를 설정하였다.

사용한 Domain ID:

ROS_DOMAIN_ID = 18

3. Turtlesim 실행

turtlesim 노드를 다음 명령어로 실행하였다.

```
ros2 run turtlesim turtlesim_node
```

turtlesim_node 를 실행하면 화면 중앙에 거북이가 생성되고, /turtle1/cmd_vel Topic 으로 전달되는 속도 메시지에 따라 움직인다.

4. 원 그리기

원은 거북이가 앞으로 이동하면서 동시에 회전하도록 설정하였다.

사용한 주요 값

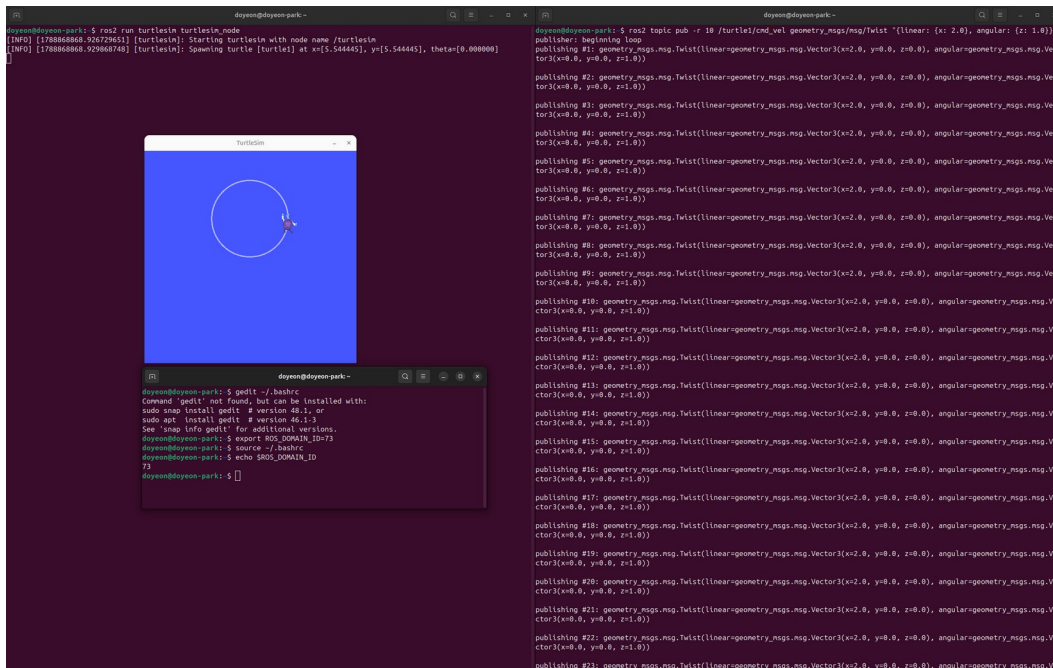
linear.x = 2.0

angular.z = 1.0

사용한 명령어 :

```
ros2 topic pub -r 10 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0}, angular: {z: 1.0}}"
```

linear.x 값을 사용하여 거북이를 앞으로 이동시키고, 동시에 angular.z 값을 주어 계속 회전하게 하였다. 두 동작이 동시에 이루어지면서 원 모양의 이동 경로가 만들어졌다.



거북이가 일정한 곡률로 이동하여 원을 그리는 것을 확인하였다.

5. 사각형 그리기

사각형 직진과 약 90 도 회전을 반복하는 방식으로 구현하였다.

직진에 사용한 명령은 다음과 같다.

```
timeout 2 ros2 topic pub -r 10 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0}, angular: {z: 0.0}}"
```

회전은 처음에 높은 각속도를 사용하였으나, 실행할 때마다 회전 각도에 오차가 발생하였다.

따라서 회전 속도를 낮춰 오차를 줄이기 위해 다음과 같이 설정하였다.

angular.z = 0.2
회전 시간 $\approx$ 7.71 초

```
timeout 7.71s ros2 topic pub -r 20 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0}, angular: {z: 0.2}}"
```

직진과 회전을 총 4 번 반복하여 사각형을 완성하였다.

직진 → 회전

직진 → 회전

직진 → 회전

직진 → 회전

```
deyeon@deyeon-park:~$ ros2 run turtlesim turtlesim_node
[INFO] [178887522.88025847] [turtlesim]: Starting turtlesim with node name /turtlesim
[INFO] [178887522.88359786] [turtlesim]: Spawning turtle (turtle1) at x:[5.54445], y:[5.54445], theta:[0.00000]
deyeon@deyeon-park:~$ ros2 run turtlesim turtlesim_node
[INFO] [178887200.41543621] [turtlesim]: Starting turtlesim with node name /turtlesim
[INFO] [178887200.51674072] [turtlesim]: Spawning turtle (turtle1) at x:[5.54445], y:[5.54445], theta:[0.00000]

deyeon@deyeon-park:~$ timeout 2 ros2 topic pub -r 10 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0}, angular: {z: 0.0}}"
ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0}, angular: {z: 0.0}}"
timeout 7.71s ros2 topic pub -r 20 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0}, angular: {z: 0.22}}"
ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0}, angular: {z: 0.0}}"
timeout 2 ros2 topic pub -r 10 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0}, angular: {z: 0.0}}"
ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0}, angular: {z: 0.0}}"
timeout 7.71s ros2 topic pub -r 20 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0}, angular: {z: 0.22}}"
ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0}, angular: {z: 0.0}}"
timeout 2 ros2 topic pub -r 10 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0}, angular: {z: 0.0}}"
ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0}, angular: {z: 0.22}}"
timeout 7.71s ros2 topic pub -r 20 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0}, angular: {z: 0.0}}"
ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0}, angular: {z: 0.22}}"
timeout 2 ros2 topic pub -r 10 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0}, angular: {z: 0.0}}"/turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0}, angular: {z: 0.0}}
publisher: beginning loop
publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0))
publishing #2: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0))
publishing #3: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0))
publishing #4: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0))
publishing #5: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0))
publishing #6: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0))
publishing #7: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0))
publishing #8: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0))
publishing #9: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0))
publishing #10: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0))
publishing #11: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0))
publishing #12: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0))
publisher: beginning loop
publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0))
publisher: beginning loop
publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.22))
publishing #2: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.22))
publishing #3: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.22))
publishing #4: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.22))
```

회전 속도를 낮춘 뒤 각도 오차가 줄어들었고, 네 변이 거의 직각을 이루는 사각형을 그릴 수 있었다.

6. 삼각형 그리기

정삼각형은 한 변을 이동한 후 약 120 도씩 회전하도록 하였다.

직진은 사각형과 동일하게 사용하였다.

timeout 2 ros2 topic pub -r 10 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0}, angular: {z: 0.0}}"

회전 속도는 사각형에서 안정적으로 동작했던 0.2 를 그대로 사용하고, 회전 시간만 증가시켰다.

angular.z = 0.2

회전 시간 ≈ 10.328 초

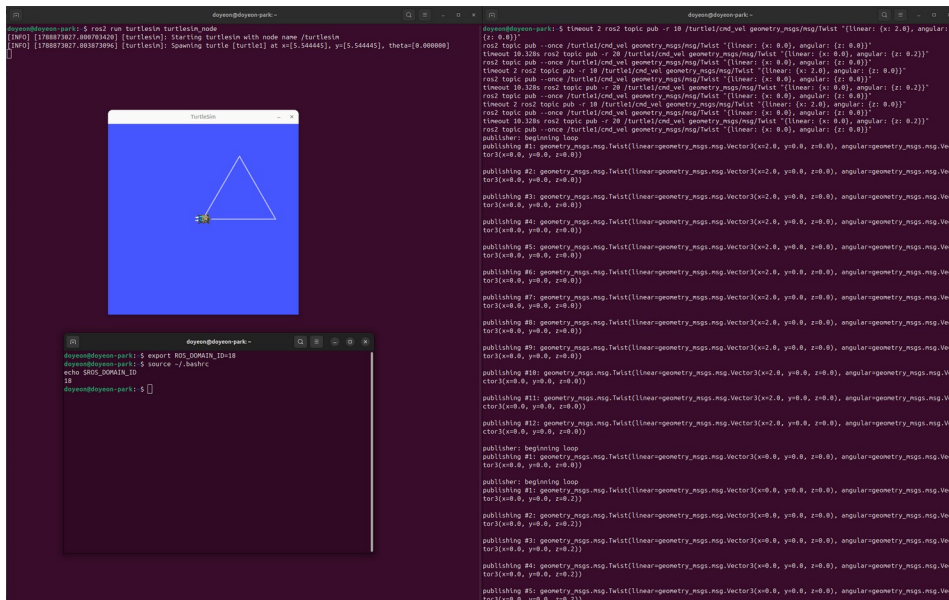
timeout 10.328s ros2 topic pub -r 20 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0}, angular: {z: 0.2}}"

을 세 번 반복하였다.

직진 $\rightarrow$ 120 도 회전

직진 $\rightarrow$ 120 도 회전

직진 $\rightarrow$ 120 도 회전



세 변이 거의 동일한 길이를 가지며 시작점 근처에서 닫히는 삼각형을 그릴 수 있었다.

7. 문제점 및 해결

과제를 진행하면서 가장 큰 문제는 회전 각도가 일정하지 않은 것이었다.

처음에는 높은 angular.z 값을 사용하여 짧은 시간 동안 회전 시켰다. 그러나 timeout 을 이용한 실행 시간이 항상 완전히 동일하지 않아, 같은 명령을 사용해도 회전 결과가 조금씩 달라졌다.

이를 해결하기 위해 회전 속도를 1.4 에서 0.2 로 낮췄다.

회전 속도가 느려지자 실행시간에서 발생하는 작은 차이가 최종 회전 각도에 미치는 영향도 줄어들어 사각형과 삼각형을 더 정확하게 그릴 수 있었다.